

Optimized Rank Sort for Encrypted Real Numbers

Seunghu Kim¹, Eymen Ünay², Ayse Yilmazer-Metin², and Hyung Tae Lee¹

¹ School of Computer Science and Engineering, Chung-Ang University, Republic of Korea
{rlatmdgn77, hyungtaelee}@cau.ac.kr

² Department of Computer Engineering, Istanbul Technical University, Turkey
{unay19, yilmazerayse}@itu.edu.tr

Abstract. Sorting arrays encrypted under fully homomorphic encryption remains a fundamental challenge due to the high cost of private comparisons and the incompatibility of conventional sorting algorithms with the encrypted domain. Recently, Hong et al. (IEEE Transactions on Information Forensics and Security, 2021) proposed a k -way sorting network tailored to encrypted real numbers, but its reliance on multiple comparison stages incurs substantial multiplicative depth and significant bootstrapping overhead, even for modest array sizes.

In this work, we propose a novel rank-based sorting algorithm for encrypted real numbers that performs only a single comparison stage, thereby eliminating the need for bootstrapping operations. Our empirical evaluation demonstrates that the proposed method significantly outperforms the k -way approach for small to medium array sizes ($n \leq 1024$), achieving a $46.91\times$ speedup at $n = 256$ with a total runtime of 79 seconds. Furthermore, we examine the recent matrix-based rank sort method by Mazzone et al. (USENIX Security '25) and show that integrating our optimized rank construction improves its efficiency. Specifically, we achieve $1.77\times$ and $2.43\times$ performance gains for $n = 128$ and $n = 512$, respectively.

Keywords: Blind sort, rank sort, sorting encrypted array, fully homomorphic encryption, CKKS

1 Introduction

Outsourcing computational tasks to untrusted servers has become increasingly prevalent in modern computing environments. However, this trend raises significant concerns about data confidentiality. Fully homomorphic encryption (FHE) offers a compelling solution by enabling arbitrary computations on encrypted data without revealing underlying plaintext, thereby ensuring end-to-end privacy. Despite its theoretical appeal, FHE-based computation remains computationally intensive, posing practical limitations on a wide range of applications.

Sorting encrypted data is one of the most challenging tasks and plays a central role in many privacy-sensitive applications. For instance, secure auction [16] must determine the highest bid without revealing individual bids, and privacy-preserving recommendation system [34] and k -nearest neighbor (k -NN) inferences [30] rely on efficient sorting of encrypted user preferences or distances. However, the high cost of sorting under FHE has limited the deployment of these services in real-world settings.

Motivated by this challenge, several studies have explored how to sort encrypted array efficiently. While comparison-based algorithms such as merge sort are known to achieve optimal $O(n \log n)$ complexity for arrays of size n in the plaintext domain [22], directly applying them to encrypted data is far from straightforward. The main obstacle is that the server performing the computation cannot observe intermediate comparison outcomes. This limitation complicates divide-and-conquer strategies, effectively degrading all algorithms to their worst case performance. For instance, merge sort becomes impractical since merging requires access to comparison results, and quick sort suffers from the inability to identify pivot positions, as highlighted in [9].

Consequently, most research has focused on deterministic and data-oblivious approaches, such as sorting networks. Classical examples include bitonic sort and odd-even merge sort [4], which requires $O(n \log^2 n)$ comparisons and can be implemented under FHE. Nevertheless, their sequential comparison stages result

in deep circuits, which in turn demand larger cryptographic parameters or frequent bootstrapping—both contributing to severe computational overhead. Although the k -way sorting network [31], later adapted for real numbers in [19], reduces the number of comparisons to $O(k \log_k^2 n)$, it still requires multiple bootstrapping operations even for moderate n , resulting in substantial computational overhead.

To address circuit depth limitations, Çetin et al. [5, 7] introduced rank sort, which computes the rank of each element using all pairwise comparisons and then reconstructs the sorted array accordingly. This approach avoids recursive comparisons and reduces depth. However, a naive rank computation entails n^2 comparisons. While single-instruction multiple-data (SIMD) based techniques were adapted to improve efficiency, existing methods [7, 20, 27] are often limited to binary or integer domains, with limited applicability to real numbers.

In this paper, we propose an efficient rank-based sorting algorithm tailored for real numbers under the CKKS FHE scheme [11]. The central innovation of our work is an optimized rank construction technique that requires only a single comparison stage when the array size n satisfies $n^2 \leq s$, where s denotes the number of slots available in CKKS. This optimization drastically reduces computational overhead and eliminates the need for bootstrapping in many practical scenarios, especially for small to medium-sized arrays.

The key contributions of this work are three-fold:

- *Optimized Rank Sort Algorithm:* We propose a novel rank-based sorting algorithm for encrypted real numbers under the CKKS FHE scheme. The algorithm leverages efficient packing and comparison techniques to minimize costly homomorphic operations. Details are presented in Sections 3 and 4.
- *Enhancement of Matrix-Based Rank Sort:* We revisit and enhance the matrix-based rank sort algorithm recently and independently proposed in [26], reducing the number of costly homomorphic operations for small n . Furthermore, for large inputs with $n^2 > s$, our method removes the overhead incurred by subarray partitioning and result recombination. The enhanced method is presented in detail in Section 5.
- *Implementation and Empirical Evaluation:* We implement our algorithm along with two baselines— k -way sorting [19] and matrix sort [26]—using their official open-source implementations. Our rank sort completes sorting a 64-element array in 35.79 seconds, achieving a speedup of $35.46\times$ and $1.85\times$ over the k -way and matrix sort methods, respectively. We further integrate our rank construction technique into the matrix sort framework, resulting in a $2.43\times$ speedup for 512-element arrays. Experimental comparisons are discussed in Section 6.

We emphasize that our proposed method achieves superior performance compared to existing approaches, particularly for small to medium-sized arrays. Therefore, it is especially well-suited for applications that require sorting up to approximately 1000 elements—such as secure auctions and personalized private recommendation systems—rather than large-scale encrypted database sorting.

2 Preliminaries

In this section, we introduce the CKKS FHE scheme [11] and related concepts that serve as building blocks for our proposed method. We begin by presenting the notations utilized throughout the paper.

Notations. For an algorithm A , the notation $a \leftarrow A$ indicates that a is an output generated by A . The symbol $\odot$ is used to denote component-wise multiplication between two matrices or arrays. When we apply $+$ and $-$ to arrays, we consider them as component-wise operations. We define $\rho(\mathbf{a}, i)$ as the circular shift of an array $\mathbf{a}$ by i positions, where a positive (resp. negative) index means left (resp. right)-side rotation. When applying the ρ operation to a matrix, the matrix is considered as a 1-dimensional tensor, flattened by concatenating each row sequentially. To indicate encrypted objects, we use the prefix **ct**, e.g., **ct.a** is an encryption of the array $\mathbf{a}$.

2.1 The CKKS FHE Scheme

The CKKS scheme [11] is an FHE scheme that supports real and complex numbers as plaintexts. Like other FHE schemes, the CKKS scheme enables packing techniques and cryptographic SIMD operations [33],

allowing multiple real values to be encrypted into a single ciphertext. In this context, we treat a plaintext message as a vector, where homomorphic operations on ciphertexts correspond to component-wise operations on the underlying plaintexts. Specifically, we denote the encryption of a vector $\mathbf{m}$ as $c = \text{Enc}_{pk}(\mathbf{m})$, with decryption satisfying $\mathbf{m} \approx \text{Dec}_{sk}(c)$ due to the inherent noise in the CKKS scheme, which does not strictly separate plaintext from error. We now introduce the key homomorphic operations supported by the CKKS scheme, which serve as fundamental components of our proposed algorithm.

- Addition: Given two ciphertexts c_1 and c_2 encrypting the plaintext vectors $\mathbf{m}_1$ and $\mathbf{m}_2$, respectively, it returns a ciphertext c_{Add} such that $\text{Add}(c_1, c_2) = c_{\text{Add}}$, where $\text{Dec}_{sk}(c_{\text{Add}}) \approx \mathbf{m}_1 + \mathbf{m}_2$.
- Subtraction: Given two ciphertexts c_1 and c_2 encrypting the plaintext vectors $\mathbf{m}_1$ and $\mathbf{m}_2$, respectively, it returns a ciphertext c_{Sub} such that $\text{Sub}(c_1, c_2) = c_{\text{Sub}}$, where $\text{Dec}_{sk}(c_{\text{Sub}}) \approx \mathbf{m}_1 - \mathbf{m}_2$.
- Multiplication: Given two ciphertexts c_1 and c_2 encrypting the plaintext vectors $\mathbf{m}_1$ and $\mathbf{m}_2$, respectively, it outputs a ciphertext c_{Mult} such that $\text{Mult}(c_1, c_2) = c_{\text{Mult}}$, where $\text{Dec}_{sk}(c_{\text{Mult}}) \approx \mathbf{m}_1 \odot \mathbf{m}_2$.
Plaintext-ciphertext multiplication is similarly defined. Given a ciphertext c_1 and a plaintext $\mathbf{m}_2$, it returns a ciphertext c_{PMult} such that $\text{Dec}_{sk}(c_{\text{PMult}}) \approx \mathbf{m}_1 \odot \mathbf{m}_2$.
- Rotation: Given a ciphertext c_1 encrypting a plaintext vector $\mathbf{m}_1$, and a rotation index l , it operates as $\text{Rot}(c_1, l) = c_{\text{Rot}}$, where $\text{Dec}_{sk}(c_{\text{Rot}}) \approx \rho(\mathbf{m}_1, l)$.

In the CKKS scheme, a vector of length $N/2$ can be packed into a single ciphertext, where N is the ring dimension (assumed to be a power of 2). As discussed in [10], shorter vectors can also be packed using sparse packing. Specifically, when encoding a vector of length n such that n divides $N/2$, the vector is replicated $N/(2n)$ times. Accordingly, arrays with $n < N/2$ are considered to be sparsely packed and, throughout this paper, are interpreted as repeated $N/(2n)$ times.

2.2 Private Comparison of Real Numbers

Comparison between encrypted real numbers under the CKKS scheme is commonly realized by evaluating a polynomial approximation of the sign function. We evaluate $\text{Comp}(x, y) = (\text{Sign}(x - y) + 1)/2$, where the sign function Sign is defined as

$$\text{Sign}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{otherwise.} \end{cases}$$

Several works [12, 23] have investigated the efficient evaluation of this non-polynomial function using compositions of polynomial approximations. The method proposed in [12] utilizes two families of polynomials, $f_n(x)$ and $g_n(x)$, where $f_n(x)$ is defined as

$$f_n(x) = \sum_{i=0}^n \frac{1}{4^i} \binom{2i}{i} x(1-x^2)^i.$$

The polynomial $g_n(x)$ is a $(2n+1)$ -degree polynomial optimized for faster convergence. (For a detailed description, readers may refer to [12].) This leads to an approximation of the sign function as $\text{Sign}(x) \approx f_n^{(d_f)} \circ g_n^{(d_g)}(x)$, where d_f and d_g are the numbers of compositions of f_n and g_n , respectively.

In this paper, we use polynomials with $n = 3$, both of which have degree 7 and can be evaluated with a multiplicative depth of $\lceil \log_2 7 \rceil = 3$ using the Paterson-Stockmeyer method [28]. Under this configuration, the sign evaluation requires a total multiplicative depth of $3d_f + 3d_g + 1$.

3 Overview of Our Rank Sort

In this section, we present a high-level overview of our approach, along with an outline of the optimization strategy.

3.1 Overview of Our Proposed Algorithm

Our proposed method consists of two main phases: rank construction and rotation index checking.

Rank Construction In this phase, we construct a rank vector $\mathbf{rk}$, where each entry $\mathbf{rk}[i]$ denotes the number of elements smaller than the i -th element. The rank is computed by summing up all pairwise comparison results as follows:

$$\mathbf{rk} = \sum_{i=1}^{n-1} \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, i)) \quad (1)$$

where Comp represents the SIMD comparison operation between two arrays and n is the size of the array. Precisely, the comparison result $\mathbf{c} \leftarrow \text{Comp}(\mathbf{a}, \mathbf{b})$ is computed as follows: For each $0 \leq i \leq n-1$,

$$\mathbf{c}[i] = \begin{cases} 1 & \text{if } \mathbf{a}[i] > \mathbf{b}[i] \\ \frac{1}{2} & \text{if } \mathbf{a}[i] = \mathbf{b}[i] \\ 0 & \text{otherwise.} \end{cases}$$

Rotation Index Checking Using the rank vector $\mathbf{rk}$, this phase rearranges the array elements into the ascending order sorted array **sorted**. Note that we only consider arrays with distinct elements in this paper, which ensures that $\mathbf{rk}$ forms a bijective mapping to indices $\{0, 1, \dots, n-1\}$. Using this property, we have

$$\mathbf{sorted}[\mathbf{rk}[i]] = \mathbf{a}[i] \text{ for } 0 \leq i < n. \quad (2)$$

To better understand this relationship, for $0 \leq j < n$, we can rewrite Eq. (2) as

$$\mathbf{sorted}[j] = \mathbf{a}[i \mid \mathbf{rk}[i] = j]. \quad (3)$$

In Eq. (3), i represents the source position and j represents the target position of $\mathbf{a}[i]$ in the sorted array. This rearrangement can be expressed through rotation operations

$$\mathbf{sorted}[j] = \rho(\mathbf{a}, i - j)[j] \quad (4)$$

where $\mathbf{rk}[i] = j$. In essence, $\mathbf{a}[i]$ should be shifted by $i - \mathbf{rk}[i]$ to be sorted.

To enable SIMD compatibility, we define the index vector $\mathbf{ind} = [0 \ 1 \ \dots \ n-1]$ and construct the rotation index vector $\mathbf{rotind} = \mathbf{ind} - \mathbf{rk}$, where each entry indicates the required rotation amount for sorting. Then, Eq. (4) can be reformulated using $\mathbf{rotind}$ as

$$\mathbf{sorted}[j] = \rho(\mathbf{a}, k)[j] \quad (5)$$

if $\mathbf{rotind}[j] = k$ for $-n < k < n$. From Eq. (5), the entire **sorted** array is evaluated as

$$\mathbf{sorted} = \sum_{i=-n+1}^{n-1} \rho(\mathbf{a} \odot (\mathbf{rotind} == i^n), i)$$

where i^n denotes the length- n vector with all entries equal to i , and $(\mathbf{rotind} == i^n)$ is the indicator vector with entries equal to 1 where the condition $\mathbf{rotind}[p] = i$ is satisfied, and 0 otherwise. To obtain this indicator vector, we compute $\text{Sinc}(\mathbf{rotind} - i^n)$ for $-n < i < n$, where the frequency-scaled Sinc function is defined as

$$\text{Sinc}(x) = \begin{cases} 1 & \text{if } x = 0, \\ \frac{\sin(\pi x)}{\pi x} & \text{otherwise} \end{cases}$$

for a real number x and is applied component-wise as $\text{Sinc}(\mathbf{x}) = (\text{Sinc}(\mathbf{x}[1]), \dots, \text{Sinc}(\mathbf{x}[n]))$ for a vector $\mathbf{x}$ of length n . Since rotation indices for a length- n vector are congruent modulo n , we define

$$\text{DoubledSinc}(x) = \text{Sinc}(x) + \text{Sinc}(x + n)$$

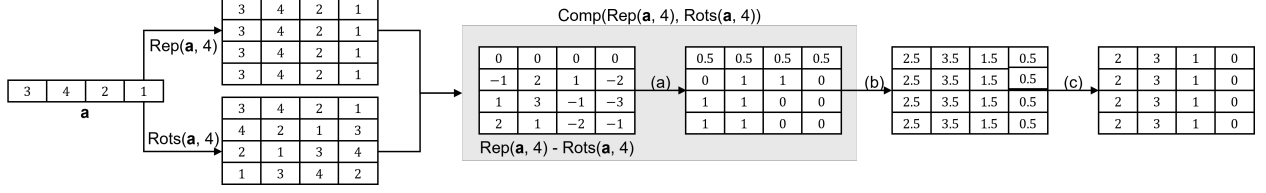


Fig. 1. Toy Example of Rank Construction ($n = 4$): An input array $\mathbf{a}$ of length 4 is first encoded using 4-fold replication, denoted as $\text{Rep}(\mathbf{a}, 4)$. Using $\text{Rep}(\mathbf{a}, 4)$ together with $\text{Rots}(\mathbf{a}, 4)$, we construct the 4-fold replicated form of the rank vector $\mathbf{rk}$, i.e., $\text{Rep}(\mathbf{rk}, 4)$. Specifically, (a) applies the function $(\text{Sign}(x) + 1)/2$ entry-wise to evaluate pairwise comparisons; (b) performs component-wise summation of row vectors to aggregate the comparison results; and (c) subtracts 0.5 entry-wise to compensate for self-comparisons.

to handle equivalent rotations for a real number x and similarly define

$$\text{DoubledSinc}(\mathbf{x}) = (\text{DoubledSinc}(\mathbf{x}[1]), \dots, \text{DoubledSinc}(\mathbf{x}[n]))$$

for a length- n vector $\mathbf{x}$. The function creates a masking vector $\text{DoubledSinc}(\mathbf{rotind} - i^n)$ that identifies entries that require an i -position rotation. Finally, the complete sorting process is expressed as

$$\text{sorted} = \sum_{i=0}^{n-1} \rho(\mathbf{a} \odot \text{DoubledSinc}(\mathbf{rotind} - i^n), i). \quad (6)$$

We remark that while this paper focuses on sorting in ascending order, the proposed framework can be naturally extended to support descending order as well.

3.2 Optimization via Loop Vectorization

The aforementioned approach requires n evaluations of both Comp and DoubledSinc approximation polynomials, incurring substantial computational costs in the encrypted domain. To optimize this, the sequential operations are transformed into SIMD operations through batching the polynomial evaluations.

First, we define $\text{Rep}(\mathbf{a}, n)$ as the n -times replicated form of array $\mathbf{a}$ and $\text{Rots}(\mathbf{a}, n)$ as the concatenation of n successive rotations of $\mathbf{a}$. These operations are expressed as

$$\begin{aligned} \text{Rep}(\mathbf{a}, n) &= \overbrace{[\mathbf{a} \dots \mathbf{a}]}^{n \text{ times}}, \\ \text{Rots}(\mathbf{a}, n) &= [\mathbf{a} \ \rho(\mathbf{a}, 1) \ \rho(\mathbf{a}, 2) \dots \rho(\mathbf{a}, n-1)]. \end{aligned}$$

Using these definitions, the comparison operation is reduced to a single evaluation of $\text{Comp}(\text{Rep}(\mathbf{a}, n), \text{Rots}(\mathbf{a}, n))$. This generates an $n \times n$ comparison matrix, where the i -th row represents the comparison between $\mathbf{a}$ and $\rho(\mathbf{a}, i-1)$. Aggregating the n comparisons via component-wise summation of row vectors (hereafter referred to as *row vector summation*) yields the rank vector in its n -fold replicated form. We illustrate a simple example of this rank construction procedure in Fig. 1.

Rotation index checking can be optimized in a similar manner. We use $\text{Rep}(\mathbf{rotind}, n)$ and a sequential indices vector of length- n^2 $\mathbf{seq}$, defined as $\mathbf{seq}[i] = \lfloor i/n \rfloor$ for $0 \leq i < n^2$. Namely, we define $\mathbf{seq} = [0^n \ 1^n \dots (n-1)^n]$ where each block i^n denotes a length- n vector consisting of the value i repeated n times. Then, we evaluate

$$\tilde{\mathbf{a}} = \text{Rep}(\mathbf{a}, n) \odot \text{DoubledSinc}(\text{Rep}(\mathbf{rotind}, n) - \mathbf{seq}).$$

$\tilde{\mathbf{a}}$ is a length- n^2 vector comprising n concatenated blocks of length n each. In the k -th block, only entries requiring a rotation of k positions retain their values, while all others become zero. By considering each block

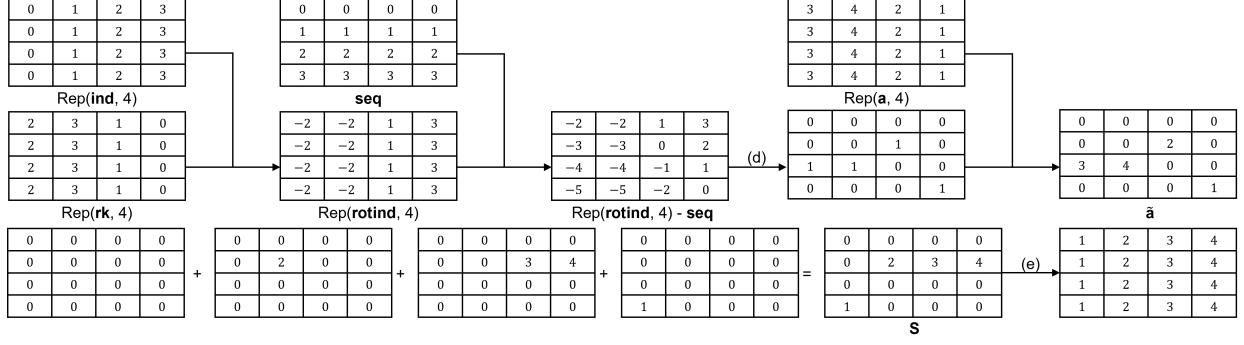


Fig. 2. Toy Example of Rotation Index Checking ($n = 4$): Given $\text{Rep}(\text{rk}, 4)$ from the rank construction phase and the index plaintext, we compute the 4-fold replicated form of rotind , where $\text{rotind} = \text{ind} - \text{rk}$. (d) evaluates the **DoubledSinc** function on $\text{Rep}(\text{rotind}, 4) - \text{seq}$, producing a masking vector used to generate $\tilde{\mathbf{a}}$. The bottom row of the figure corresponds to Eq. (7), where the k -th component contains the k -th row of $\tilde{\mathbf{a}}$ rotated by k ($0 \leq k < 4$). Aggregating these yields $\mathbf{S}$. Finally, (e) aggregates the row vectors via component-wise summation to complete the sorting process.

as a row, it can also be represented as an $n \times n$ matrix

$$\tilde{\mathbf{a}} = \begin{bmatrix} \mathbf{a} \odot \text{DoubledSinc}(\text{rotind} - 0^n) \\ \vdots \\ \mathbf{a} \odot \text{DoubledSinc}(\text{rotind} - i^n) \\ \vdots \\ \mathbf{a} \odot \text{DoubledSinc}(\text{rotind} - (n-1)^n) \end{bmatrix}.$$

Subsequently, we compute the $n \times n$ sorted matrix $\mathbf{S}$ (which also can be viewed as a length- n^2 array) as

$$\mathbf{S} = \sum_{k=0}^{n-1} \rho(\tilde{\mathbf{a}} \odot \mathbf{b}^{(k)}, k) \quad (7)$$

where $\mathbf{b}^{(k)} \in \{0, 1\}^{n^2}$ is a block-extracting vector of length n^2 , defined as

$$\mathbf{b}^{(k)}[i] = \begin{cases} 1 & \text{if } kn \leq i < (k+1)n, \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

for $0 \leq k < n$. Let $\tilde{\mathbf{a}}_k$ denote the k -th row of $\tilde{\mathbf{a}}$. When the extracting vector $\mathbf{b}^{(k)}$ is multiplied with $\tilde{\mathbf{a}}$, it isolates $\tilde{\mathbf{a}}_k$ from $\tilde{\mathbf{a}}$, allowing it to be rotated by k positions as in Eq. (6). Summing all n rows of the matrix $\mathbf{S}$ yields the sorted array. Fig. 2 describes a toy example of this process when $n = 4$.

4 Our Rank Sort Method

In this section, we formalize our rank sort algorithm which operates on encrypted arrays, building upon the high-level overview provided in the previous section.

4.1 Sub-Algorithms for Our Rank Sort Algorithm

We begin by introducing three sub-algorithms that serve as building blocks for the main rank sort algorithm.

Generation of Concatenated Rotations Recall that our rank construction method leverages vectorized comparisons using replicated arrays, denoted by $\text{Rep}(\mathbf{a}, n)$, and concatenated rotations, denoted by $\text{Rots}(\mathbf{a}, n)$. The sparsely packed and encrypted array ct.a , which is a CKKS ciphertext of the plaintext vector $\mathbf{a}$, can be naturally interpreted as $\text{ct.Rep}(\mathbf{a}, n)$. From this representation, $\text{ct.Rots}(\mathbf{a}, n)$ can be efficiently generated. Algorithm 1 illustrates this procedure, which generates the n -fold concatenated rotations of $\mathbf{a}$ in the encrypted domain. To this end, we use the extracting vector $\mathbf{b}^{(k)}$, defined in Eq. (8).

Algorithm 1 GenRots(ct.a)

Input: ct.a, which is a sparsely packed ciphertext of the length- n array $\mathbf{a}$

Output: ct.rots, which is a ciphertext of $\text{Rots}(\mathbf{a}, n)$

```

1: ct.rots  $\leftarrow$  ct.0
2: for  $i = 0$  to  $n - 1$  do
3:   ct.a(i)  $\leftarrow$  Rot(ct.a,  $i$ )
4:   ct.ã(i)  $\leftarrow$  PMult(ct.a(i),  $\mathbf{b}^{(i)}$ )
5:   ct.rots  $\leftarrow$  Add(ct.rots, ct.ã(i))
6: end for
7: return ct.rots

```

Correctness of Algorithm 1. We note that the underlying plaintext of ct.a is equivalent to $\text{Rep}(\mathbf{a}, n)$. Let $\mathbf{a}[i : j]$ denote the sub-array of $\mathbf{a}$ containing elements from indices i to j . By the definition of Rep , we have the following relation: For $0 \leq i < n$,

$$\text{Rep}(\mathbf{a}, n)[i \cdot n : i \cdot n + (n - 1)] = \mathbf{a}.$$

From Eq. (8), the extracting operation yields

$$\text{Rep}(\mathbf{a}, n) \odot \mathbf{b}^{(k)} = [0^{k \cdot n} \ \mathbf{a} \ 0^{(n-k-1) \cdot n}].$$

In the algorithm, Line 3 computes a ciphertext $\text{ct.a}^{(i)}$ that encrypts

$$\mathbf{a}^{(i)} = \rho(\text{Rep}(\mathbf{a}, n), i) = \text{Rep}(\rho(\mathbf{a}, i), n)$$

where $\rho(\mathbf{a}, i)$ is the circular left shift of array $\mathbf{a}$ by i positions, for a non-negative integer i . In Line 4, since PMult performs the Hadamard product on the underlying plaintexts, the resulting ciphertext encrypts

$$\tilde{\mathbf{a}}^{(i)} = \mathbf{a}^{(i)} \odot \mathbf{b}^{(i)} = [0^{i \cdot n} \ \rho(\mathbf{a}, i) \ 0^{(n-i-1) \cdot n}].$$

From Line 5, ct.rots is finally computed as an encryption of

$$\mathbf{rots} = \sum_{i=0}^{n-1} \tilde{\mathbf{a}}^{(i)} = [\rho(\mathbf{a}, 0) \ \dots \ \rho(\mathbf{a}, n-1)] = \text{Rots}(\mathbf{a}, n).$$

Therefore, the output ct.rots is a ciphertext of $\text{Rots}(\mathbf{a}, n)$.

Efficiency and Optimization of Algorithm 1. A naive method requires n Rot operations, which can be optimized via the baby-step giant-step approach [18]. Let n_p be the number of precomputed rotations, assumed to divide n . The set of precomputed ciphertexts is given by $\{\text{ct.a}^{(j)} = \text{Rot}(\text{ct.a}, j)\}_{j=0}^{n_p-1}$. Our optimized method computes

$$\text{GenRots}(\text{ct.a}, n) = \sum_{i=0}^{(n/n_p)-1} \text{Rot}\left(\sum_{j=0}^{n_p-1} \alpha_{i,j}, n_p \cdot i\right) \quad (9)$$

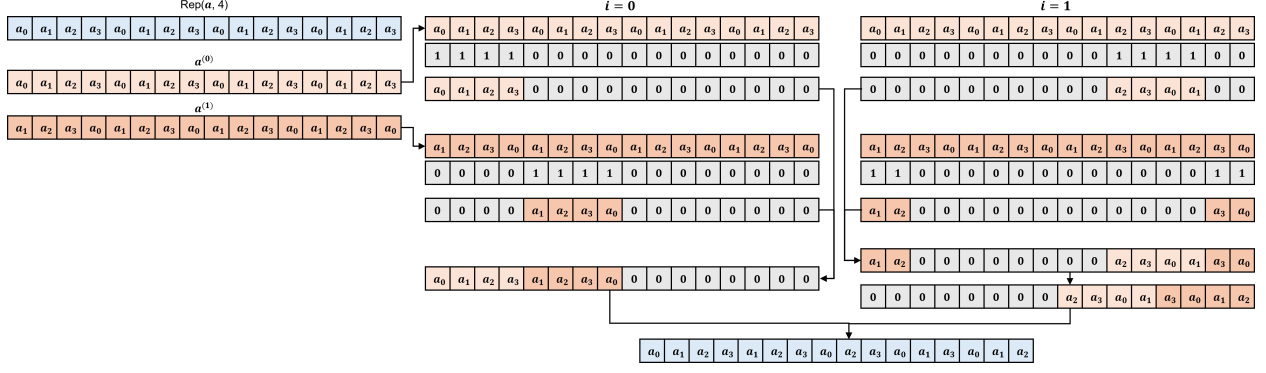


Fig. 3. Visualization of Baby-Step Giant-Step Optimization in GenRots: This illustrates how the baby-step giant-step optimization is applied in GenRots, with parameters $n = 4$, $n_p = 2$ precomputed rotations, and $r = 4$ replications. The precomputed rotations $\mathbf{a}^{(0)} = \mathbf{a}$ and $\mathbf{a}^{(1)} = \rho(\mathbf{a}, 1)$ are reused across iterations over the index i , along with appropriate extracting plaintexts. As a result, the output is $\text{Rots}(\mathbf{a}, 4)$.

where homomorphic addition (Add) is expressed using summation. Here, $\alpha_{i,j} = \text{PMult}(\text{ct}.\mathbf{a}^{(j)}, \rho(\mathbf{b}^{(n_p \cdot i + j)}, -n_p \cdot i))$. This remains correct because, for any index $k = n_p \cdot i + j$, we can decompose $\text{Rot}(\text{ct}.\mathbf{a}, k)$ as

$$\text{Rot}(\text{Rot}(\text{ct}.\mathbf{a}, j), n_p \cdot i).$$

Additionally, we rearrange PMult and Rot using the relation

$$\text{PMult}(\text{Rot}(X, \delta), Y) = \text{Rot}(\text{PMult}(X, \rho(Y, -\delta)), \delta).$$

Thus, our optimization produces the same result as the original algorithm. The total number of required rotations, $n_p + \frac{n}{n_p}$, is minimized when $n_p = \frac{n}{n_p}$. Setting $n_p \approx \sqrt{n}$ optimizes the process to approximately $2\sqrt{n}$ rotations. When applying this optimization, we invoke GenRots with $\mathbf{pr} = \{\text{ct}.\mathbf{a}^{(j)}\}_{j=0}^{n_p-1}$ as an additional parameter, enabling efficient reuse of precomputed rotations. As a result, GenRots performs only n/n_p rotations internally. To illustrate this optimization more clearly, Fig. 3 presents a visualization of the GenRots procedure in a simplified case.

Generalization of Algorithm 1. For an array of size n , the algorithm requires n^2 slots. Suppose that $n < (N/2) < n^2$ where N is the ring dimension of the CKKS scheme and $N/2$ is the maximum slot capacity. That is, all components of the array can be encrypted into a single ciphertext; however a single ciphertext cannot accommodate n^2 slots. In this case, we use $B = n^2/(N/2)$ ciphertexts, each of which contains $r = (N/2)/n$ replicated arrays or concatenated rotations. The generalized GenRots algorithm takes an input array $\mathbf{a}$ along with a start rotation index i_s and an end index i_e , such that $r = i_e - i_s$, and outputs the encryption of a length- $(r \cdot n)$ array of the form

$$[\rho(\mathbf{a}, i_s) \dots \rho(\mathbf{a}, i_e - 1)].$$

For the above computation, the number of required Rot operations is r/n_p .

Rank Construction Algorithm Building on GenRots, we propose a homomorphic rank construction algorithm. Algorithm 2 takes an encrypted array as an input and produces an encrypted rank array. At its core, it evaluates approximate comparisons, computing $\text{Comp}(x, y) = (\text{Sign}(x - y) + 1)/2$ between replicated arrays and concatenated rotations. We again remark that the input $\text{ct}.\mathbf{a}$ is sparsely packed and thus it can be regarded as a CKKS ciphertext of $\text{Rep}(\mathbf{a}, n)$.

Algorithm 2 ConRank(ct.a): Rank Construction

Input: ct.a, which is a sparsely packed ciphertext of the length- n array $\mathbf{a}$

Output: ct.rk^{out}

```
1: for  $i = 0$  to  $n_p - 1$  do
2:    $\mathbf{pr}[i] \leftarrow \text{Rot}(\text{ct.a}, i)$ 
3: end for
4:  $\text{ct.rots} \leftarrow \text{GenRots}(\text{ct.a}, \mathbf{pr})$ 
5:  $\text{ct.rk}^{\text{int}} \leftarrow \text{Comp}(\text{ct.a}, \text{ct.rots})$ 
6: for  $i = 1$  to  $\log_2 n$  do ▷ Row Vector Summation
7:    $\text{ct.rk}^{\text{int}} \leftarrow \text{Add}(\text{ct.rk}^{\text{int}}, \text{Rot}(\text{ct.rk}^{\text{int}}, n^2/2^i))$ 
8: end for
9:  $\text{ct.rk}^{\text{out}} \leftarrow \text{Sub}(\text{ct.rk}^{\text{int}}, 0.5)$ 
10: return ct.rkout
```

Correctness of Algorithm 2. The algorithm ConRank outputs a ciphertext encrypting the n -replication of the rank array $\mathbf{rk}$ defined in Eq. (1). From the correctness of GenRots, Line 4 computes a ciphertext of the vector

$$\mathbf{rots} = [\mathbf{a} \rho(\mathbf{a}, 1) \dots \rho(\mathbf{a}, n-1)].$$

In Line 5, the component-wise comparison Comp returns a ciphertext of the length- n^2 array $\mathbf{rk}^{\text{int}}$, which can be interpreted as an $n \times n$ matrix

$$\mathbf{rk}^{\text{int}} = \begin{bmatrix} \text{Comp}(\mathbf{a}, \mathbf{a}) \\ \vdots \\ \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, n-1)) \end{bmatrix}.$$

The row vector summation performed in Lines 6–8 aggregates all rows, resulting in a ciphertext of

$$\mathbf{rk}^{\text{int}} = [\sum_{i=0}^{n-1} \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, i))]^n$$

where the superscript n denotes the n -fold concatenation of the resulting row vector. Notably, the self-comparison $\text{Comp}(\mathbf{a}, \mathbf{a}) = [0.5 \dots 0.5] \in \mathbb{R}^n$ contributes 0.5 to each element's own rank. Line 9³ compensates for this by subtracting the self-comparison, yielding

$$\mathbf{rk}^{\text{out}} = [\sum_{i=1}^{n-1} \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, i))]^n = \text{Rep}(\mathbf{rk}, n).$$

Therefore, the final output ct.rk^{out} is a ciphertext encrypting $\text{Rep}(\mathbf{rk}, n)$.

Efficiency of Algorithm 2. The algorithm requires approximately $2\sqrt{n} + \log_2 n$ Rot operations, where $n_p = \sqrt{n}$ accounts for the number of precomputed rotations, $n/n_p = \sqrt{n}$ from the call to GenRots, and $\log_2 n$ from the row vector summation.

Generalization of Algorithm 2. Suppose a ciphertext ct.a of a length- n array $\mathbf{a}$ is given, where $n < (N/2) < n^2$ for a ring dimension N in the CKKS scheme. Since we assume that ct.a is sparsely packed, it can be interpreted as an encryption of $\text{Rep}(\mathbf{a}, r)$, where $r = (N/2)/n$. We then apply the GenRots and Comp algorithms to $B = n^2/(N/2)$ blocks, each starting from the same ciphertext ct.a. For the i -th block ($i = 1, \dots, B$), GenRots takes $i_s = (i-1)r$ and $i_e = ir-1$, and outputs a ciphertext of

$$\mathbf{rots}^{(i)} = [\rho(\mathbf{a}, (i-1)r) \dots \rho(\mathbf{a}, ir-1)].$$

The corresponding comparison result for the i -th block is given by

$$[\text{Comp}(\mathbf{a}, \rho(\mathbf{a}, (i-1)r)), \dots, \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, ir-1))].$$

³ In Line 9, we interpret the constant 0.5 as its encrypted form and subtract it homomorphically.

Aggregating these over all i yields the intermediate rank matrix

$$\mathbf{rk}^{\text{int}} = \begin{bmatrix} \sum_{i=1}^B \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, (i-1)r)) \\ \vdots \\ \sum_{i=1}^B \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, ir-1)) \end{bmatrix}.$$

After applying the row vector summation, we obtain

$$\begin{aligned} \mathbf{rk}^{\text{out}} &= \left[\sum_{i=1}^B \sum_{j=0}^{r-1} \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, (i-1)r + j)) \right]^r \\ &= \left[\sum_{i=0}^{n-1} \text{Comp}(\mathbf{a}, \rho(\mathbf{a}, i)) \right]^r \end{aligned}$$

where the superscript r denotes the r -fold concatenation of the resulting row vector. Finally, subtracting the self-comparison contribution of 0.5 from each entry yields an encryption of the r -fold replication of the rank vector $\mathbf{rk}$.

Precomputed rotations $\mathbf{pr} = \{\text{ct}.\mathbf{a}^{(j)} = \text{Rot}(\text{ct}.\mathbf{a}, j)\}_{j=0}^{n_p-1}$ are performed once and reused across all B executions of GenRots. Specifically, the same optimization in Eq. (9) applies, with a shift by i_s in the rotation

$$\text{GenRots}(\text{ct}.\mathbf{a}, i_s, i_e) = \sum_{i=0}^{(r/n_p)-1} \text{Rot}\left(\sum_{j=0}^{n_p-1} \alpha_{i,j}, i_s + n_p \cdot i\right)$$

where $\alpha_{i,j} = \text{PMult}(\text{ct}.\mathbf{a}^{(j)}, \rho(\mathbf{b}^{(n_p \cdot i + j)}, -i_s - n_p \cdot i))$ and homomorphic addition (Add) is expressed using summation. In this case, the number of required Rot operations is $n_p + B(r/n_p) + \log_2 r$, where n_p rotations are used to compute $\mathbf{pr}$, $B(r/n_p)$ rotations arise from the giant-step across all B blocks, and $\log_2 r$ rotations are incurred in Line 7 of Algorithm 2 for the row vector summation. By setting $n_p \approx B \cdot (r/n_p)$, we obtain $n_p + B(r/n_p) \approx 2\sqrt{B \cdot r} = 2\sqrt{n}$ since $n_p^2 \approx B \cdot r = n$. Thus, the total number of required Rot operations is approximately $2\sqrt{n} + \log_2 r$.

Shift Row Algorithm To complete the sorting, we need to evaluate Eq. (7). The length- n^2 array $\tilde{\mathbf{a}}$ consists of n partitions, where the i -th partition contains entries of $\mathbf{a}$ that are to be rotated by i positions. When viewed as an $n \times n$ matrix, $\tilde{\mathbf{a}}$ is interpreted such that the i -th row is rotated left by i positions, thereby realizing a homomorphic row-wise shift. This operation is described in Algorithm 3, which treats the plaintext array $\mathbf{a}$, corresponding to the input ciphertext $\text{ct}.\mathbf{a}$, as an $n \times n$ matrix and outputs $\text{ct}.\mathbf{S}$, a ciphertext of the matrix $\mathbf{S}$ defined in Eq. (7).

Algorithm 3 ShiftRow(ct.a)

Input: ct.a, a ciphertext of a length- n^2 array $\mathbf{a}$

Output: ct.S

```

1: ct.S  $\leftarrow$  ct.0
2: for  $i = 0$  to  $n - 1$  do
3:   ct.a(i)  $\leftarrow$  PMult(ct.a, b(i))
4:   ct.a(i)  $\leftarrow$  Rot(ct.a(i), i)
5:   ct.S  $\leftarrow$  Add(ct.S, ct.a(i))
6: end for
7: return ct.S
```

Correctness of Algorithm 3. Since PMult and Rot respectively perform component-wise multiplication and cyclic rotation on the plaintext, Line 4 computes a ciphertext $\text{ct}.\mathbf{a}^{(i)}$ encrypting $\mathbf{a}^{(i)} = \rho(\mathbf{a} \odot \mathbf{b}^{(i)}, i)$. By summing over all $i = 0, \dots, n-1$, we obtain $\mathbf{S} = \sum_{i=0}^{n-1} \rho(\mathbf{a} \odot \mathbf{b}^{(i)}, i)$, which corresponds to Eq. (7).

Efficiency of Algorithm 3. A naive implementation uses n Rot operations, but this can be reduced to $2\sqrt{n}$ via the baby-step giant-step method: Given $\mathbf{pr} = \{\mathbf{ct.a}^{(j)} = \text{Rot}(\mathbf{ct.a}, j)\}_{j=0}^{n_p-1}$ and $\beta_{i,j} = \text{PMult}(\mathbf{ct.a}^{(j)}, \rho(\mathbf{b}^{(n_p \cdot i + j)}, j))$, we compute

$$\text{ShiftRow}(\mathbf{ct.a}) = \sum_{i=0}^{(r/n_p)-1} \text{Rot}\left(\sum_{j=0}^{n_p-1} \beta_{i,j}, n_p \cdot i\right)$$

where the homomorphic addition (Add) is represented using summation notation.

Generalization of Algorithm 3. To generalize beyond the square $n \times n$ case, we extend the input to an $r \times n$ matrix, where $r < n$, and denote this algorithm by $\text{ShiftRow}(\mathbf{ct.a}, i_s)$. In this setting, the first row is rotated by i_s , and each subsequent row is rotated by an incrementally increasing index, up to $i_s + r - 1$. This generalization leverages the same mechanism as the $n \times n$ case, applying Rot and PMult operations, and incurs a total of approximately $2\sqrt{r}$ Rot operations.

4.2 Our Proposed Rank Sort Algorithm

We now present our proposed rank sort algorithm. Building upon the sub-algorithms introduced in Section 4.1, the complete procedure is described in Algorithm 4.

Algorithm 4 Sort(ct.a): Our Rank Sort Algorithm

Input: $\mathbf{ct.a}$, which is a sparsely packed ciphertext of the length- n array $\mathbf{a}$

Output: $\mathbf{ct.S}$, which is a ciphertext of the sorted array

```

1:  $\mathbf{ct.rk} \leftarrow \text{ConRank}(\mathbf{ct.a})$ 
2:  $\mathbf{ct.rotind} \leftarrow \text{Sub}(\mathbf{ct.ind}, \mathbf{ct.rk})$ 
3:  $\mathbf{ct.r} \leftarrow \text{Sub}(\mathbf{ct.rotind}, \mathbf{ct.seq})$ 
4:  $\mathbf{ct.r} \leftarrow \text{DoubledSinc}(\mathbf{ct.r})$ 
5:  $\mathbf{ct.\tilde{a}} \leftarrow \text{Mult}(\mathbf{ct.a}, \mathbf{ct.r})$ 
6:  $\mathbf{ct.S} \leftarrow \text{ShiftRow}(\mathbf{ct.\tilde{a}})$ 
7: for  $i = 1$  to  $\log_2 n$  do ▷ Row Vector Summation
8:    $\mathbf{ct.S} \leftarrow \text{Add}(\mathbf{ct.S}, \text{Rot}(\mathbf{ct.S}, n^2/2^i))$ 
9: end for
10: return  $\mathbf{ct.S}$ 

```

Correctness of Algorithm 4. It follows directly from the correctness of the underlying FHE scheme, along with the correctness of the sub-algorithms ConRank and ShiftRow, that Eq. (7) can be correctly evaluated as encrypted. First, Line 1 computes a ciphertext of the rank vector $\mathbf{rk}$. It also holds the followings:

- $\mathbf{ct.rotind}$ is a ciphertext of $\mathbf{ind} - \mathbf{rk}$.
- $\mathbf{ct.r}$ is a ciphertext of $\text{DoubledSinc}(\mathbf{rotind} - \mathbf{seq})$.
- $\mathbf{ct.\tilde{a}}$ is a ciphertext of $\mathbf{a} \odot \text{DoubledSinc}(\mathbf{rotind} - \mathbf{seq})$.
- After executing Line 6, $\mathbf{ct.S}$ is a ciphertext of

$$\sum_{i=0}^{n-1} \rho(\mathbf{a} \odot \text{DoubledSinc}(\mathbf{rotind} - \mathbf{seq}), i)$$

which corresponds to Eq. (7). At this point, $\mathbf{S}$ is column-wise sorted, but distributed across n rows.

Subsequently, in Lines 7–9, summing over these rows aggregates the distributed entries, yielding a ciphertext of the fully sorted array in n -fold replicated form.

Efficiency of Algorithm 4. We analyze the efficiency of our sorting method, focusing on the computational bottleneck: the required number of Rot operations and evaluations of non-polynomial functions (such as Comp

and DoubledSinc). The required number of Rot operations is $4\sqrt{n} + 2\log_2 n$, where $2\sqrt{n} + \log_2 n$ comes from ConRank, $2\sqrt{n}$ from ShiftRow and $\log_2 n$ from row vector summation. It evaluates the Comp and DoubledSinc functions once each.

In terms of circuit depth, our method requires a total of $2\text{PMult} + 1\text{Mult}$, in addition to the multiplicative depths needed for evaluating Comp and DoubledSinc functions. Specifically, one PMult arises from the ConRank algorithm and the other from ShiftRow, while the single Mult is incurred in Line 5 of Algorithm 4. As previously discussed, the multiplicative depth of Comp is given by $\lceil \log_2(2n_g + 1) \rceil \cdot d_g + \lceil \log_2(2n_f + 1) \rceil \cdot d_f + 1$, where the parameters n_g , n_f , d_g , and d_f are selected based on the desired comparison precision. For the DoubledSinc function, the degree n_s of the polynomial approximation grows with the array size n . When evaluated using the Paterson–Stockmeyer method, its multiplicative depth is approximately $\log_2 n_s$.

Sorting Large Arrays. Based on the generalization, our rank sort algorithm supports large arrays where $n^2 > N/2$. In this case, n -fold replication is infeasible, so we use r -fold replication with $r = (N/2)/n$, processing the input over B iterations such that $Br = n$.

The rank construction correctly yields an r -fold replicated **rk**. For each $i = 1, \dots, B$, we define the length- (rn) array

$$\text{seq}^{(i)} := [((i-1)r)^n \dots (ir-1)^n],$$

and compute $\mathbf{r}^{(i)} = \text{DoubledSinc}(\text{rotind} - \text{seq}^{(i)})$, forming

$$\mathbf{r}^{(i)} = \begin{bmatrix} \text{DoubledSinc}(\text{rotind} - ((i-1)r)^n) \\ \vdots \\ \text{DoubledSinc}(\text{rotind} - ((ir-1)^n)) \end{bmatrix}.$$

Let $\tilde{\mathbf{a}}^{(i)}$ denote the masked array at iteration i . Then $\text{ShiftRow}(\tilde{\mathbf{a}}^{(i)}, (i-1)r)$ returns a partially sorted array $\mathbf{S}^{(i)}$, and the sorted array $\mathbf{S}$ is obtained by aggregating over i . The required number of Rot operations is $2\sqrt{n} + 2B\sqrt{r} + 2\log_2 r$, and the Comp and DoubledSinc functions are each computed B times. Importantly, the depth cost remains the same.

5 Revisiting the Matrix-Based Rank Sort

In this section, we revisit the matrix-based rank sort method recently introduced in [26], and present an improved variant by incorporating our proposed rank construction algorithm.

5.1 Overview of the Matrix-Based Rank Sort in [26]

We review the method proposed in [26], which adopts a matrix-based approach to rank sort. Fig. 4 illustrates a toy example of this method for the case of $n = 4$. As shown in the figure, the algorithm begins by transposing the input from row-wise to column-wise layout, followed by replication of both matrices. We note that the input array must be zero-padded prior to transposition, which incurs additional overhead during replication.

After comparing two resulting square matrices, the algorithm generates a rank matrix **Rank**. It then subtracts the sequence matrix, $\mathbf{Seq} = [1^n \dots n^n]$, from **Rank**⁴ and applies the Ind_0 function, defined as $\text{Ind}_0(x) = 1$ if $x = 0$, and 0 otherwise. This operation yields a binary masking matrix, which ensures that the element ranked k is placed in the k -th row of the resulting matrix. Since the sorted values are now aligned row-wise, the final sorted array can be obtained by aggregating the columns followed by a column-to-row transposition. To this end, Algorithm 5 aggregates all columns into the first column, while zeroing out the remaining ones. This is done using the extracting vector

$$\bar{\mathbf{b}}^{(k)}[i] = \begin{cases} 1 & \text{if } k \equiv i \pmod{n} \\ 0 & \text{otherwise.} \end{cases}$$

⁴ **Rank** and **Seq** are analogous to **rk** and **seq**, respectively, but with entries in the range 1 to n rather than 0 to $n-1$.

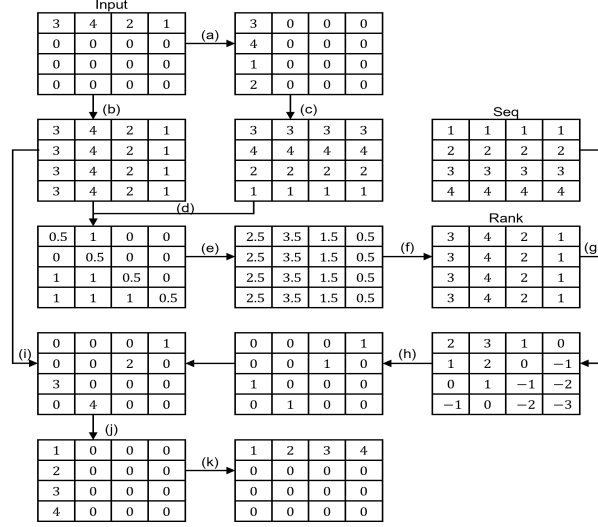


Fig. 4. Toy Example of Matrix-Based Rank Sorting ($n = 4$): (a) is row to column transposition, (b) is row replication, (c) is column replication, (d) is comparison, (e) is row vector summation, (f) is component-wise addition, (g) is **Rank** – **Seq** operation, (h) is lnd_0 function evaluation, (i) is masking input array, (j) is column sum and (k) is column to row transposition.

Next, Algorithm 6 converts the column-wise encoding into a row-encoded format using the extracting vector defined in Eq. (8). For further details, we refer the reader to [26].

Algorithm 5 SumC(ct.A): Column Summation from Algorithm 10 in [26]

Input: ct.A, a ciphertext of an $n \times n$ matrix **A**

Output: ct.C, a ciphertext of length- n vector encoded as a column, containing the sum across all columns

```

1: for  $i = 1$  to  $\log_2 n$  do
2:   ct.A  $\leftarrow$  Add(ct.A, Rot(ct.A,  $2^{i-1}$ ))
3: end for
4: ct.C  $\leftarrow$  PMult(ct.A,  $\bar{\mathbf{b}}^{(0)}$ )
5: return ct.C

```

Algorithm 6 TransC(ct.A): Column-To-Row Transposition

Input: ct.A, a ciphertext of a length- n array encoded as a single column of an $n \times n$ matrix **A**, with all other entries set to zero

Output: ct.A^T, a ciphertext of length- n input array encoded as the first row, with all other entries set to zero

```

1: for  $i = 1$  to  $\log_2 n$  do
2:   ct.A  $\leftarrow$  Add(ct.A, Rot(ct.A,  $n(n-1)/2^i$ ))
3: end for
4: ct.AT  $\leftarrow$  PMult(ct.A,  $\mathbf{b}^{(0)}$ )
5: return ct.AT

```

5.2 Improved Matrix-Based Rank Sort

While the matrix-based rank sorting method offers a conceptually elegant approach, it leaves room for optimization, particularly in rank construction. The original method incurs a total of $4 \log_2 n$ Rot operations:

$\log_2 n$ each for transposition, row replication, column replication, and row vector summation. In contrast, our method requires only $2\sqrt{n} + \log_2 n$ Rot operations under the same constraint $n^2 \leq N/2$. Since $2\sqrt{n} + \log_2 n < 4\log_2 n$ holds for $n < 64$, our approach is strictly more efficient in this range.

When $n^2 > N/2$, the efficiency gap becomes even more significant. Our approach requires about $2\sqrt{n} + \log_2 r$ Rot operations, whereas the matrix-based method requires $6t\log_2 \bar{n}$ Rot operations, in addition to the overhead of splitting and merging subarrays. Here, $t = n/\bar{n}$ denotes the number of blocks, and $\bar{n} = 2^{\lceil \log_2 \sqrt{N/2} \rceil}$ represents the size of each block.

To address these limitations, we propose an improved matrix-based method by integrating our optimized rank construction algorithm. This not only reduces the number of Rot operations but also eliminates the overheads with array partitioning. To this end, we develop auxiliary matrix algorithms that enable efficient large-array processing without the need for splitting and merging.

Binary Path Computation Algorithm 5 and Algorithm 6 place the results of summation and transposition in the first column and row, respectively. To generalize these to arbitrary positions, we introduce binary path computation in Algorithm 7, which enables targeting any specified column or row.

Given an index $j \in \{0, \dots, n-1\}$, the algorithm computes its binary representation of length $d = \log_2 n$ in most-significant-bit to least-significant-bit order. The resulting binary vector $\mathbf{p}$ corresponds to a traversal path from the root to the j -th leaf in a perfect binary tree of depth d , where each bit in $\mathbf{p}$ indicates a left (0) or right (1) branch at each level.

Algorithm 7 GetBinaryPath(j): Binary Path from Index

Input: $j \in \{0, \dots, n-1\}$ where $n = 2^d$

Output: $\mathbf{p}$, a binary path vector of length d

```

1: for  $i = 1$  to  $d$  do
2:    $q \leftarrow \lfloor j/2^{d-i} \rfloor$ 
3:    $\mathbf{p}[i-1] \leftarrow q \bmod 2$ 
4: end for
5: return  $\mathbf{p}$ 

```

Correctness of Algorithm 7. Suppose that the binary representation of the input index j is

$$j = b_0 + b_1 \cdot 2 + b_2 \cdot 2^2 + \dots + b_{d-1} \cdot 2^{d-1}$$

where $b_i \in \{0, 1\}$. Then, for any $1 \leq i \leq d$, we can write

$$j = 2^{d-i} \cdot q + r \quad \text{with} \quad q = b_{d-i} + b_{d-i+1} \cdot 2 + \dots + b_{d-1} \cdot 2^{i-1}$$

and so the $(i-1)$ -th entry of $\mathbf{p}$ is $\mathbf{p}[i-1] = b_{d-i}$.

More Flexible Matrix Algorithms We present generalized matrix algorithms in which the target column (or row) for storing the result depends on the direction of rotation—either left or right. Given a target index j , its binary path determines the corresponding sequence of rotations. Algorithm 8 performs column summation and stores the result in the j -th column.

Correctness of Algorithm 8. Let $\mathbf{A}_k$ and $\mathbf{C}_k$ denote the k -th row of matrices $\mathbf{A}$ and $\mathbf{C}$, respectively. Then, it suffices to prove that the algorithm outputs a ciphertext of $\mathbf{C}$ satisfying

$$\mathbf{C}_k[j] = \sum_{m=0}^{n-1} \mathbf{A}_k[m]$$

Algorithm 8 SumCT(ct.A, j): Summation to Target Column

Input: ct.A, a ciphertext of an $n \times n$ matrix **A** and an index j ($0 \leq j < n$) indicating the target column

Output: ct.C, a ciphertext of an $n \times n$ matrix where the j -th column contains the sum across all columns for each row

```
1: p  $\leftarrow$  GetBinaryPath( $j$ )
2: step  $\leftarrow n/2$ 
3: for  $i = 1$  to  $\log_2 n$  do
4:   if p[ $i-1$ ] = 1 then
5:     ct.A  $\leftarrow$  Add(ct.A, Rot(ct.A,  $-\text{step}$ ))
6:   else
7:     ct.A  $\leftarrow$  Add(ct.A, Rot(ct.A,  $\text{step}$ ))
8:   end if
9:   step  $\leftarrow \text{step}/2$ 
10: end for
11: ct.C  $\leftarrow$  PMult(ct.A,  $\bar{\mathbf{b}}^{(j)}$ )
12: return ct.C
```

for the given target index j and each $0 \leq k < n$.

To that end, let $\mathbf{A}^{(i)}$ be the intermediate plaintext matrix of ct.A at iteration i , where $\mathbf{A}^{(0)} = \mathbf{A}$. During each iteration $i \in \{1, \dots, \log_2 n\}$, we maintain an active interval $[m_s^{(i)}, m_e^{(i)})$ that tracks the valid column indices involved in partial summation, initialized as $[m_s^{(0)}, m_e^{(0)}) = [0, n)$. We define the step size as $\text{step} = n/2^i$ and update the interval and matrix entries as follows:

$$\begin{aligned} m_s^{(i)} &\leftarrow m_s^{(i-1)} + \mathbf{p}[i-1] \cdot \text{step}, \\ m_e^{(i)} &\leftarrow m_e^{(i-1)} - (1 - \mathbf{p}[i-1]) \cdot \text{step}, \\ \mathbf{A}_k^{(i)}[m] &\leftarrow \mathbf{A}_k^{(i-1)}[m] + \mathbf{A}_k^{(i-1)}[(m \pm \text{step}) \bmod n], \end{aligned}$$

for all $m \in [m_s^{(i)}, m_e^{(i)})$, where the sign in $m \pm \text{step}$ is determined by $\mathbf{p}[i-1]$ of the binary path for j .

At iteration i , each m in the active interval accumulates 2^i entries from the original matrix

$$\mathbf{A}_k^{(i)}[m] = \sum_{u=0}^{2^i-1} \mathbf{A}_k^{(0)}[(m + u \cdot n/2^i) \bmod n].$$

After $\log_2 n$ iterations, the active interval converges to

$$m_s^{(\log_2 n)} = \sum_{i=1}^{\log_2 n} \mathbf{p}[i-1] \cdot n/2^i = j \quad \text{and} \quad m_e^{(\log_2 n)} = j + 1.$$

Thus, for each row k , we have

$$\mathbf{A}_k^{(\log_2 n)}[j] = \sum_{u=0}^{n-1} \mathbf{A}_k^{(0)}[(j + u) \bmod n] = \sum_{m=0}^{n-1} \mathbf{A}_k[m].$$

Finally, multiplying the extracting vector $\bar{\mathbf{b}}^{(j)}$ ensures that only the j -th column is retained, with all others zeroed out.

Efficiency of Algorithm 8. SumCT requires $\log_2 n$ Rot and Add operations, along with one PMult.

Similarly, Algorithm 9 generalizes Algorithm 6: Given a ciphertext of an $n \times n$ matrix with non-zero entries only in the k -th column, it outputs a ciphertext of a matrix in which the k -th row contains the transposed values of that column.

Algorithm 9 TransCT(ct.A, k): Transpose to Target Row

Input: ct.A, a ciphertext of a length- n array encoded as the k -th column in an $n \times n$ matrix $\mathbf{A}$, with all other entries set to zero

Output: ct.B, a ciphertext of a length- n array encoded as a k -th row in an $n \times n$ matrix $\mathbf{A}^T$

```
1:  $\mathbf{p} \leftarrow \text{GetBinaryPath}(k)$ 
2:  $\text{step} \leftarrow n(n-1)/2$ 
3: for  $l = 1$  to  $\log_2 n$  do
4:   if  $\mathbf{p}[l-1] = 1$  then
5:      $\text{ct.A} \leftarrow \text{Add}(\text{ct.A}, \text{Rot}(\text{ct.A}, -\text{step}))$ 
6:   else
7:      $\text{ct.A} \leftarrow \text{Add}(\text{ct.A}, \text{Rot}(\text{ct.A}, \text{step}))$ 
8:   end if
9:    $\text{step} \leftarrow \text{step}/2$ 
10: end for
11:  $\text{ct.B} \leftarrow \text{PMult}(\text{ct.A}, \mathbf{b}^{(k)})$ 
12: return ct.B
```

Correctness of Algorithm 9. We interpret the matrix $\mathbf{A}$ in the input ciphertext ct.A as a flattened array of length n^2 , and assume that only the k -th column can contain nonzero entries, with all other entries set to zero. Then, it suffices to show that the matrix $\mathbf{B}$, likewise interpreted as a length- n^2 array in the output ciphertext ct.B, satisfies

$$\mathbf{B}[n \cdot j + i] = \begin{cases} \mathbf{A}[n \cdot i + j] & \text{if } j = k, \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

for $i, j \in \{0, 1, \dots, n-1\}$.

Algorithm 9 performs Rot and Add operations over iterations indexed by l to propagate the nonzero entries in the k -th column of $\mathbf{A}$. According to Eq. (10), correctly transposing the (i, k) -th entry requires rotating a left rotation by $(i - k)(n - 1)$ positions.

Let $S = \{\delta_l\}_{l=1}^d$ denote the set of rotation offsets, determined by $\mathbf{p}$ and step in Algorithm 9, with $d = \log_2 n$. From Lines 4–7 in Algorithm 9, each offset is determined by

$$\delta_l = (1 - 2\mathbf{p}[l-1]) \cdot \frac{n}{2^l} \cdot (n-1)$$

where the factor $(1 - 2\mathbf{p}[l-1])$ determines the rotation direction. For any integer $0 \leq m < n$ with binary representation $m = \sum_{l=1}^d \mathbf{m}[l-1] \cdot 2^{d-l}$, let $S_m = \{\delta_l \mid \mathbf{m}[l-1] = 1\}$. Then, the cumulative offset corresponding to subset S_m is

$$\begin{aligned} \sum_{\delta \in S_m} \delta &= \sum_{l=1}^d \mathbf{m}[l-1] \cdot \delta_l \\ &= \sum_{l=1}^d \mathbf{m}[l-1] \cdot (1 - 2\mathbf{p}[l-1]) \cdot \frac{n}{2^l} \cdot (n-1) \\ &= (n-1) \left(\sum_{l=1}^d \mathbf{m}[l-1] \cdot 2^{d-l} - 2 \sum_{l=1}^d \mathbf{m}[l-1] \cdot \mathbf{p}[l-1] \cdot 2^{d-l} \right) \\ &= (n-1)(m - 2 \cdot (m \wedge k)) \\ &= (n-1)(m \oplus k - k) \end{aligned}$$

where $\wedge$ and $\oplus$ denote bitwise AND and XOR operations, respectively, and k is the index corresponding to $\mathbf{p}$. The third equality holds since $n = 2^d$ and the last equality holds since $m \oplus k = m + k - 2 \cdot (m \wedge k)$.

This expression enumerates all rotation distances applied to the nonzero entries in the k -th column of $\mathbf{A}$ during the recursive Rot and Add steps, since $\{m \oplus k | m = 0, \dots, n-1\} = \{0, \dots, n-1\}$ for any fixed k . In particular, it includes the specific offset $(i-k)(n-1)$ required for placing entry (i, k) into its transposed position (k, i) . Furthermore, since the nonzero entries are initially spaced by n positions and each offset $(\bar{m}-k)(n-1)$ is distinct for $\bar{m} = 0, \dots, n-1$, the rotated entries are placed in disjoint positions, avoiding overlap. Finally, applying the extracting vector $\mathbf{b}^{(k)}$ zeroes out all rows except the k -th, completing the targeted transposition.

Efficiency of Algorithm 9. TransCT requires $\log_2 n$ Rot and Add operations, along with one PMult.

Revisited Sorting Algorithm We first present the revisited sorting method for the case $n^2 < N/2$ in Algorithm 10, and then discuss its generalization to larger arrays, where our flexible matrix algorithms become essential for scalability.

Algorithm 10 Sort(ct.a): Matrix Based Rank Sort, Revisited

Input: ct.a, which is a sparsely packed ciphertext of the length- n array $\mathbf{a}$

Output: ct.S, a ciphertext of the sorted array

- 1: $\text{ct.rk} \leftarrow \text{ConRank}(\text{ct.a})$
 - 2: $\text{ct.M} \leftarrow \text{Sub}(\text{ct.rk}, \text{seq})$
 - 3: $\text{ct.M} \leftarrow \text{Sinc}(\text{ct.M})$
 - 4: $\text{ct.A} \leftarrow \text{Mult}(\text{ct.a}, \text{ct.M})$
 - 5: $\text{ct.C} \leftarrow \text{SumCT}(\text{A}, 0)$
 - 6: $\text{ct.S} \leftarrow \text{TransCT}(\text{C}, 0)$
 - 7: **return** ct.S
-

Correctness of Algorithm 10. Our ConRank correctly produces an n -fold replication of the rank vector $\mathbf{rk}$, forming an $n \times n$ matrix. After applying the Sinc function, the i -th row of the resulting matrix $\mathbf{M}$ satisfies $\mathbf{M}_i[j] = 1$ if $\mathbf{rk}_i[j] = i$, and 0 otherwise. Multiplying this mask with the input array $\mathbf{a}$ yields a matrix $\mathbf{A}_i$ in which each row $\mathbf{A}_i$ contains only the element of $\mathbf{a}$ that should be placed at the i -th position in the sorted output, with all other entries set to zero. Column-wise summation by SumCT then aggregates these elements into the first column of matrix $\mathbf{C}$, which holds the sorted array. A final transposition step by TransCT transforms this column into row form, yielding the encrypted sorted output.

Efficiency of Algorithm 10. In the algorithm, the input array is sparsely packed, eliminating the need for replication. It requires a total of $2\sqrt{n} + 3\log_2 n$ Rot operations: $2\sqrt{n} + \log_2 n$ for ConRank, and $\log_2 n$ each for SumCT and TransCT. This reduces the original $6\log_2 n$ Rot operations and improves efficiency for small input sizes ($n < 128$).

In terms of multiplicative depth, the algorithm requires $3\text{PMult} + 1\text{Mult}$, excluding the cost of non-polynomial function approximation. Notably, replacing the previous Ind_0 function with Sinc results in a shallower depth.

Sorting Large Arrays. When $n^2 > N/2$, we divide the input into subarrays of size $\bar{n}$, where $\bar{n}^2 \leq N/2$ and $\bar{n} \mid n$. Let $t = n/\bar{n}$ denote the number of subarrays. For each $0 \leq i < t$, we define a sequence matrix $\text{seq}^{(i)}$ of size $\bar{n} \times \bar{n}$ as

$$\text{seq}^{(i)}[l] = i \cdot \bar{n} + \lfloor l/\bar{n} \rfloor, \quad \text{for } 0 \leq l < \bar{n}^2.$$

This matrix $\text{seq}^{(i)}$ serves to identify entries whose ranks lie in the interval $[i \cdot \bar{n}, (i+1) \cdot \bar{n})$, enabling us to extract the correct elements for the i -th sorted subarray. Repeating this process across all t subarrays yields the fully sorted array.

To this end, we precompute the following rotations:

$$\mathbf{rk}^{(j)} = \rho(\mathbf{rk}, j \cdot \bar{n}), \quad \mathbf{a}^{(j)} = \rho(\mathbf{a}, j \cdot \bar{n}),$$

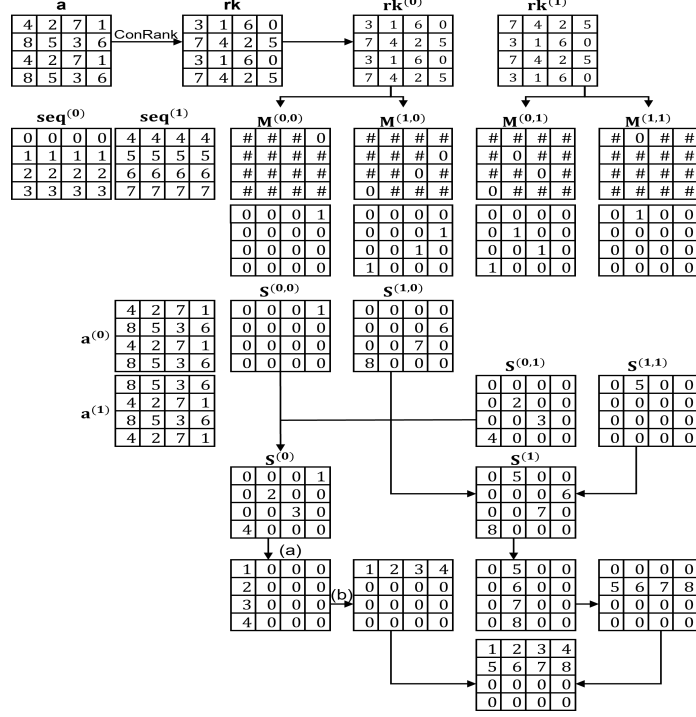


Fig. 5. Toy Example of Revisited Method for Sorting Large Arrays ($n = 8$, $N/2 = 16$): This illustrates the case $N/2 = 16 < n^2 = 64$, where the input array $\mathbf{a}$ is replicated $r = 2$ times. Each masking matrix $\mathbf{M}^{(i,j)} = \text{Sinc}(\mathbf{rk}^{(j)} - \mathbf{seq}^{(i)})$ is derived using $\mathbf{rk}^{(j)} = \rho(\mathbf{rk}, 4j)$, with nonzero entries marked as $\#$. These matrices mask $\mathbf{a}^{(j)}$, and the results are summed over i to form $\mathbf{S}^{(i)}$. (a) and (b) illustrate column summation and transposition steps (Algorithm 8 and Algorithm 9), after which sorting is completed by aggregating the resulting rows.

for $j = 0, \dots, t-1$. Here, both $\mathbf{rk}$ and $\mathbf{a}$ are in r -fold replicated form, with $n \cdot r = N/2$. Thus, the segment $\mathbf{rk}[j \cdot \bar{n} : (j+1) \cdot \bar{n}]$ corresponds to the ranks of the j -th subarray. Rotating by $j \cdot \bar{n}$ aligns the ranks of the j -th subarray with the index range defined by $\mathbf{seq}^{(i)}$.

For each pair (i, j) in t^2 total iterations, we compute

$$\mathbf{M}^{(i,j)} = \text{Sinc}(\mathbf{rk}^{(j)} - \mathbf{seq}^{(i)}), \quad \mathbf{A}^{(i,j)} = \mathbf{a}^{(j)} \odot \mathbf{M}^{(i,j)}.$$

Let $\mathbf{A}^{(i)} = \sum_{j=0}^{t-1} \mathbf{A}^{(i,j)}$. Each matrix $\mathbf{A}^{(i,j)}$ contains elements from $\mathbf{a}$ whose ranks fall within $[i \cdot \bar{n}, (i+1) \cdot \bar{n})$, and zeros elsewhere. To complete sorting, we apply the following matrix operations to place the i -th sorted subarray into the i -th row of $\mathbf{S}$:

$$\text{ct.C} \leftarrow \text{SumCT}(\text{ct.A}, i), \quad \text{ct.S} \leftarrow \text{TransCT}(\text{ct.C}, i).$$

A toy example illustrating this process is provided in Fig. 5.

The generalized algorithm requires $2\sqrt{n} + \log_2 r + 2\log_2 \bar{n} + 2t$ Rot operations in total: $2\sqrt{n} + \log_2 r$ for ConRank, $\log_2 \bar{n}$ each for SumCT and TransCT, and $2t$ for precomputed rotations. This represents a substantial reduction from the original $8t\log_2 \bar{n}$ cost. Furthermore, by employing generalized matrix operations, the algorithm eliminates the overhead associated with subarray handling, particularly those involving Rot and PMult.

6 Efficiency Comparison

This section presents experimental results comparing the efficiency of our proposed sorting method with existing approaches, including k -way sorting [19] and matrix sort [26].

Experimental Settings. Our implementation⁵ uses the OpenFHE library with version 1.1.4 [2]. All experiments were conducted on a system with an Intel Xeon Gold 6226R CPU (2.90GHz) and 256GB RAM running Ubuntu 22.04 LTS. We employed the CKKS scheme with a ring dimension $N = 2^{17}$ and a scaling factor of 2^{40} for all methods, except k -way sorting, which used 2^{59} to preserve precision during bootstrapping. For fair comparison, we applied identical sign approximation parameters (d_g, d_f) across all methods.

Each reported result is the average of 10 trials. Input arrays were randomly generated with values in $[0, 1]$, ensuring minimum pairwise separation to consistent accuracy evaluation. Execution time and sorting error (L1 distance from the correctly sorted array) were recorded for each method.

Table 1. Efficiency Comparison of Matrix-Based Rank Sort [26], Our Rank Sort and Revisited Matrix-Based Rank Sort

n	(d_g, d_f)	Matrix-Based Rank Sort in [26]					Our Rank Sort				Revisited Matrix-Based Rank Sort			
		L	Time (s)	Error (log2)		L	Time (s)	Error (log2)		L	Time (s)	Error (log2)		L
				Max	Avg			Max	Avg			Max	Avg	
4	(2, 2)	31	28.18	-11.53	-13.20	23	15.57	-19.83	-20.32	24	15.38	-20.82	-21.58	
8	(2, 2)	35	38.86	-26.27	-27.23	24	18.68	-22.13	-23.08	25	17.88	-20.38	-21.06	
16	(2, 2)	35	42.33	-13.80	-17.70	25	21.29	-20.47	-21.85	25	19.74	-19.22	-20.72	
32	(3, 2)	42	61.26	-23.40	-24.42	28	29.39	-16.55	-18.34	29	26.50	-18.67	-20.34	
64	(3, 2)	42	66.18	-16.74	-21.84	29	35.79	-16.57	-18.44	30	33.91	-17.30	-19.21	
128	(3, 2)	46	80.95	-20.65	-22.30	30	51.64	-15.24	-17.21	31	45.83	-16.09	-18.41	
256	(4, 2)	49	94.23	-19.33	-21.17	34	79.00	-10.55	-13.08	44	78.54	-19.67	-21.52	
512	(4, 2)	57	368.00	-18.52	-20.51	35	164.96	-12.35	-14.60	47	151.67	-18.97	-20.78	
1024	(5, 2)	60	427.33	-17.85	-19.72	39	856.08	-9.54	-12.14	50	378.64	-18.20	-19.82	

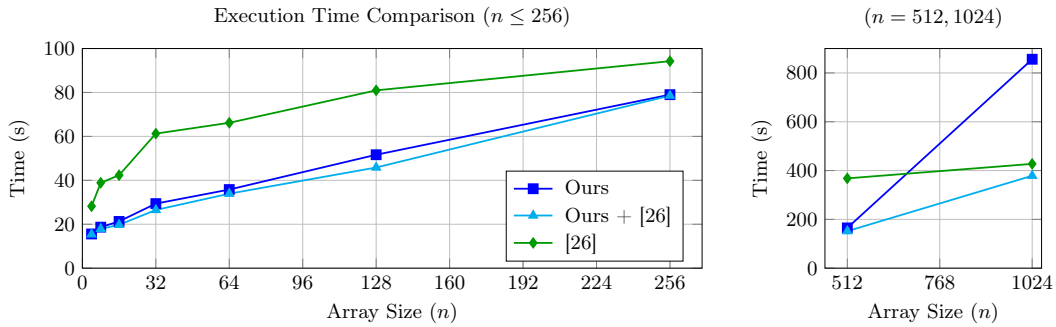


Fig. 6. Comparison of Execution Times: Performance measured in seconds relative to array size n across three algorithms: our proposed method, the matrix-based rank sort from [26], and a hybrid approach combining our rank construction method with [26]. The left panel shows results for smaller arrays ($n \leq 256$), while the right panel presents larger arrays ($n = 512, 1024$) where multiple non-polynomial function evaluations are required.

⁵ Available at <https://github.com/oksuman/FHE-Sorting.git>

Performance Evaluation. Table 1 presents the execution times of three sorting algorithms: the original matrix sort algorithm from [26], our proposed rank sort algorithm described in Section 4, and our revisited matrix sort algorithm introduced in Section 5. The corresponding execution time trends are also visualized in Figure 6. As shown in the table, both our rank sort and revisited matrix sort algorithms outperform the original matrix sort for small to medium array sizes (i.e., $n \leq 512$). For instance, when $n = 128$, our rank sort and revisited matrix sort achieve speedups of $1.57\times$ and $1.77\times$, respectively, compared to the original matrix sort. For $n = 512$, these improvements increase $2.23\times$ and $2.43\times$, respectively. However, for large arrays, the performance advantage of our rank sort diminishes. Specifically, at $n = 1024$, the original matrix sort outperforms our rank sort by a factor of $2.00\times$. This performance degradation stems from the increasing complexity of accurately approximating the DoubledSinc function used in our method, whose depth advantage over the Ind_0 function diminishes as n grows. Nevertheless, our revisited matrix sort algorithm maintains its advantage even for large arrays, achieving a $1.12\times$ speedup over the original matrix sort at $n = 1024$.

Table 2. Experimental result of the k -way Algorithm [19]

k	n	(d_g, d_f)	Time (s)	Error (log2)	
				Max	Avg
2	4	(2, 2)	89.67	-46.58	-47.55
	8	(2, 2)	248.28	-16.26	-17.22
	16	(2, 2)	469.48	-14.26	-15.51
	32	(3, 2)	892.21	-14.12	-15.57
	64	(3, 2)	1269.22	-13.05	-14.80
	128	(4, 2)	2412.87	-12.58	-13.77
	256	(4, 2)	3706.52	-10.93	-12.83
	512	(4, 2)	4467.57	-9.32	-11.47
3	1024	(5, 2)	5581.48	-8.90	-9.59
	9	(2, 2)	156.70	-15.50	-16.87
	27	(3, 2)	591.42	-14.59	-16.21
	81	(3, 2)	1141.49	-13.72	-15.42
	243	(4, 2)	2635.10	-11.46	-13.05
5	729	(5, 2)	3775.01	-9.71	-10.81
	25	(3, 2)	651.45	-14.88	-16.40
	125	(3, 2)	1748.94	-10.41	-12.47
5	625	(5, 2)	4509.51	-10.81	-12.67

* $L = 40$, $\Delta = 59$

Table 2 reports the experimental results of the k -way sorting algorithm proposed in [19]. Although this method was primarily designed for large input sizes, we include it as a baseline for comparison in the context of sorting encrypted real numbers. In contrast to our bootstrapping-free approaches, the k -way sorting algorithm involves multiple comparison stages, each require bootstrapping, which introduces substantial computational overhead. As a result, its performance is significantly inferior to depth-optimized methods, particularly for small to medium-sized arrays. For instance, when $n = 1024$, our revisited matrix sort achieves a $14.74\times$ speedup over the k -way sorting algorithm with $k = 2$.

7 Related Work

In this section, we review related works on sorting encrypted arrays using FHE. Since our work focuses on arrays over real numbers, we provide more detailed descriptions of existing methods on this domain.

Sorting of Encrypted Array under FHE Aguilar-Melchor et al. [1] first explored sorting encrypted arrays using FHE, implementing bubble sort with the hcrypt library [29] based on the Smart-Vercauteran scheme [32]. Chatterjee et al. [8] later proposed lazy sort, a hybrid of bubble and insertion sort, aiming to optimize for nearly sorted inputs. However, subsequent studies [5, 15] showed that this hybrid approach does not provide any benefit since without access to intermediate comparison results, insertion sort suffers worst-case complexity even on nearly sorted data. In particular, the authors in [15] compared various sorting algorithms in the encrypted domain using the hcrypt library and demonstrated that odd-even merge sort outperforms bubble sort, insertion sort, and bitonic sort.

Çetin et al. [5] focused on reducing circuit depth in sorting networks and proposed direct and greedy sort algorithms, both based on exhaustive pairwise comparisons. Follow up works [6, 7] improved direct sort and introduced rank sort, leveraging packing and SIMD techniques. Kim et al. [21] adapted bitonic sort for FHE with SIMD support, while Narumanchi et al. [27] implemented sorting algorithms using polynomial interpolation for integer-wise comparisons. Their results showed that the bit-wise comparison method proposed by Çetin et al. [5] outperforms their integer-wise comparison method when used in sorting.

Recently, Huang et al. [20] extended direct sort to larger plaintext domains via integer-wise private comparisons [25], achieving over $10\times$ improvement over bit-wise methods. Lee et al. [24] introduced an FHE-friendly shell sort based on the randomized shell sort [17] using TFHE [13], emphasizing its in-place advantage for memory-constrained settings.

Sorting of Encrypted Real Numbers One approach to sorting real-number arrays is to encode them into bit representations. Basilakis et al. [3] scaled fixed point numbers to integers and encoded them as bits for use in the direct sort [5]. Alternatively, Chung et al. [14] proposed a continued fraction-based encoding that enables adjustable precision and low-depth comparison, also integrated into direct sort.

Another direction uses approximate FHE schemes like CKKS [11]. Hong et al. [19] proposed a sorting algorithm based on a k -way sorting network [31], leveraging polynomial sign approximations [12] for real-number comparison. Although this approach reduces comparison depth from $O(2\log_2^2 n)$ to $O(k\log_k^2 n)$, which achieves better complexity for large array sizes n when $k = 5$, it still incurs high latency from repeated comparisons and bootstrapping when $n = k^m$. More recently, a matrix-based rank sort [26] was introduced, but it requires n^2 slots, limiting scalability for array sizes.

8 Conclusion

In this paper, we presented a novel sorting algorithm for real-number arrays encrypted under the CKKS FHE scheme, based on computing the rank of each entry. At the heart of our approach is an optimized homomorphic rank construction algorithm that minimizes the number of rotation operations by leveraging rotation replication and a baby-step giant-step strategy. Our implementation and experimental evaluation demonstrate that the proposed method significantly outperforms existing approaches [19, 26] on small to medium-sized array ($n \leq 512$).

Furthermore, we revisited the matrix-based rank sort recently proposed in [26], which also relies on rank computation. By incorporating our optimized rank construction into their framework, we effectively reduce the number of homomorphic rotations and eliminate the overhead introduced by array partitioning and recombination. Experimental results confirm that this integration yields tangible performance improvements over the original method.

References

1. Aguilar-Melchor, C., Fau, S., Fontaine, C., Gogniat, G., Sirdey, R.: Recent advances in homomorphic encryption: A possible future for signal processing in the encrypted domain. *IEEE Signal Process. Mag.* **30**(2), 108–117 (2013)
2. Badawi, A.A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Quah, I., Polyakov, Y., Saraswathy, R.V., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: OpenFHE: Open-source fully homomorphic encryption library. In: Brenner, M., Costache, A., Rohloff, K. (eds.) *WHAC 2022*. pp. 53–63. ACM (2022)

3. Basilakis, J., Javadi, B.: Efficient parallel binary operations on homomorphic encrypted real numbers. *IEEE Trans. Emerg. Top. Comput.* **9**(1), 507–519 (2021)
4. Batchier, K.E.: Sorting networks and their applications. In: *AFIPS 1968*. vol. 32, pp. 307–314. ACM (1968)
5. Çetin, G.S., Doröz, Y., Sunar, B., Savas, E.: Depth optimized efficient homomorphic sorting. In: Lauter, K.E., Rodríguez-Henríquez, F. (eds.) *LATINCRYPT 2015*. LNCS, vol. 9230, pp. 61–80. Springer (2015)
6. Çetin, G.S., Savas, E., Sunar, B.: Homomorphic sorting with better scalability. *IEEE Trans. Parallel Distributed Syst.* **32**(4), 760–771 (2021)
7. Çetin, G.S., Sunar, B.: Homomorphic rank sort using surrogate polynomials. In: Lange, T., Dunkelman, O. (eds.) *LATINCRYPT 2017*. LNCS, vol. 11368, pp. 311–326. Springer (2017)
8. Chatterjee, A., Kaushal, M., Sengupta, I.: Accelerating sorting of fully homomorphic encrypted data. In: Paul, G., Vaudenay, S. (eds.) *INDOCRYPT 2013*. LNCS, vol. 8250, pp. 262–273. Springer (2013)
9. Chatterjee, A., Sengupta, I.: Sorting of fully homomorphic encrypted cloud data: Can partitioning be effective? *IEEE Trans. Serv. Comput.* **13**(3), 545–558 (2020)
10. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: Nielsen, J.B., Rijmen, V. (eds.) *EUROCRYPT 2018, Part I*. LNCS, vol. 10820, pp. 360–384. Springer (2018)
11. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017, Part I*. LNCS, vol. 10624, pp. 409–437. Springer (2017)
12. Cheon, J.H., Kim, D., Kim, D.: Efficient homomorphic comparison methods with optimal complexity. In: *Advances in Cryptology - ASIACRYPT 2020, Part II*. Lecture Notes in Computer Science, vol. 12492, pp. 221–256. Springer (2020)
13. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Cheon, J.H., Takagi, T. (eds.) *ASIACRYPT 2016, Part I*. LNCS, vol. 10031, pp. 3–33 (2016)
14. Chung, H., Kim, M., Badawi, A.A., Aung, K.M.M., Veeravalli, B.: Homomorphic comparison for point numbers with user-controllable precision and its applications. *Symmetry* **12**(5), 788 (2020)
15. Emmadi, N., Gauravaram, P., Narumanchi, H., Syed, H.: Updates on sorting of fully homomorphic encrypted data. In: *ICCCRI 2015*. pp. 19–24. IEEE Computer Society (2015)
16. Ganesh, C., Kanukurthi, B., Shankar, G.: Secure auctions in the presence of rational adversaries. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) *ACM CCS 2022*. pp. 1173–1186. ACM (2022)
17. Goodrich, M.T.: Randomized shellsort: A simple data-oblivious sorting algorithm. *J. ACM* **58**(6), 27:1–27:26 (2011)
18. Halevi, S., Shoup, V.: Faster homomorphic linear transformations in HElib. In: Shacham, H., Boldyreva, A. (eds.) *CRYPTO 2018, Part I*. LNCS, vol. 10991, pp. 93–120. Springer (2018)
19. Hong, S., Kim, S., Choi, J., Lee, Y., Cheon, J.H.: Efficient sorting of homomorphic encrypted data with k -way sorting network. *IEEE Trans. Inf. Forensics Secur.* **16**, 4389–4404 (2021)
20. Huang, H., Wang, Y., Wang, L., Ge, H., Gu, Q.: Secure word-level sorting based on fully homomorphic encryption. *J. Inf. Secur. Appl.* **71**, 103372 (2022)
21. Kim, P., Lee, Y., Yoon, H.: Sorting method for fully homomorphic encrypted data using the cryptographic single-instruction multiple-data operation. *IEICE Trans. Commun.* **99-B**(5), 1070–1086 (2016)
22. Knuth, D.E.: *The art of computer programming, Volume III*. Addison-Wesley (1998)
23. Lee, E., Lee, J., No, J., Kim, Y.: Minimax approximation of sign function by composite polynomial for homomorphic comparison. *IEEE Trans. Dependable Secur. Comput.* **19**(6), 3711–3727 (2022)
24. Lee, J., Kim, Y., No, J.: Analysis of modified shell sort for fully homomorphic encryption. *IEEE Access* **9**, 126198–126215 (2021)
25. Lu, W., Zhou, J., Sakuma, J.: Non-interactive and output expressive private comparison from homomorphic encryption. In: Kim, J., Ahn, G., Kim, S., Kim, Y., López, J., Kim, T. (eds.) *AsiaCCS 2018*. pp. 67–74. ACM (2018)
26. Mazzone, F., Everts, M.H., Hahn, F., Peter, A.: Efficient ranking, order statistics, and sorting under CKKS. To appear in *USENIX Security 2025* (2025)
27. Narumanchi, H., Goyal, D., Emmadi, N., Gauravaram, P.: Performance analysis of sorting of FHE data: Integer-wise comparison vs bit-wise comparison. In: Barolli, L., Takizawa, M., Enokido, T., Hsu, H., Lin, C. (eds.) *AINA 2017*. pp. 902–908. IEEE Computer Society (2017)
28. Paterson, M., Stockmeyer, L.J.: On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM J. Comput.* **2**(1), 60–66 (1973)
29. Perl, H., Brenner, M., Smith, M.: Poster: An implementation of the fully homomorphic smart-vercauteren cryptosystem. In: Chen, Y., Danezis, G., Shmatikov, V. (eds.) *ACM CCS 2011*. pp. 837–840. ACM (2011)

30. Shaul, H., Feldman, D., Rus, D.: Secure k-ish nearest neighbors classifier. *Proc. Priv. Enhancing Technol.* **2020**(3), 42–61 (2020)
31. Shi, F., Yan, Z., Wagh, M.D.: An enhanced multiway sorting network based on n -sorters. In: *GlobalSIP 2014*. pp. 60–64. IEEE (2014)
32. Smart, N.P., Vercauteren, F.: Fully homomorphic encryption with relatively small key and ciphertext sizes. In: Nguyen, P.Q., Pointcheval, D. (eds.) *PKC 2010*. LNCS, vol. 6056, pp. 420–443. Springer (2010)
33. Smart, N.P., Vercauteren, F.: Fully homomorphic SIMD operations. *Des. Codes Cryptogr.* **71**(1), 57–81 (2014)
34. Verma, P., Mathuria, A., Dasgupta, S.: Efficient privacy preserving top- k recommendation using homomorphic sorting. *IACR Cryptol. ePrint Arch.* p. 1564 (2022)